TABLE OF CONTENTS

Unit I

- 1. Programming Paradigms
- 2. Need for Object-Oriented Programming
- 3. Fundamentals of OOP
- 4. Namespaces
- 5. Objects and Classes
- 6. Data Members, Methods and Messages
- 7. Data Encapsulation and Information Hiding
- 8. Data Abstraction
- 9. Inheritance
- 10. Polymorphism
- 11. Benefits of OOP
- 12. Java as an Object-Oriented Language
- 13. Overview of Java Program Structure
- 14. Java Packages, JVM and JDK/JRE
- 15. Data Types and Variables
- 16. Primitive vs Reference Types
- 17. Floating-Point Numbers
- 18. Operators and Expressions
- 19. Java Class Libraries
- 20. Typical Java Development Environment and Memory Concepts
- 21. Control Statements
- 22. Unit I Case Study and Activities

Unit II

- 23. Defining a Class
- 24. Field Declaration
- 25. Method Declaration and Definition
- 26. Instantiating Objects
- 27. Accessing Class Members
- 28. Methods with Multiple Parameters
- 29. Argument Passing
- 30. Object as a Parameter
- 31. Returning Objects
- 32. Reference Variables
- 33. set/get Methods
- 34. Constructors

- 35. this Keyword
- 36. Static Methods and Members
- 37. Scope of Declaration
- 38. Method Overloading
- 39. Java API Packages
- 40. Arrays in Java
- 41. Passing Arrays to Methods
- 42. Multidimensional Arrays
- 43. Variable-Length Arguments
- 44. Command-Line Arguments
- 45. Unit II Case Study: Banking/Library
- 46. Unit II Revision and Practice

UNIT I — INTRODUCTION TO OOP CONCEPTS AND CONTROL STRUCTURE

Syllabus coverage: Programming paradigms; need of OOP; namespaces; objects; classes; data members; methods; messages; encapsulation; abstraction; information hiding; inheritance; polymorphism; benefits of OOP; Java as an OOP language; Java program structure; packages; JVM; data types; primitive/reference types; floating-point numbers; operators and expressions; Java class libraries; development environment; memory concepts; selection and iteration statements; case study.

Big picture
A programming paradigm is a way of thinking about and organizing a program. OOP models a software system as a collection of objects that contain data and behavior. Java supports this style strongly through classes, objects, encapsulation, inheritance and polymorphism.

1. Programming Paradigms

A programming paradigm is a general approach or style used to design and write programs. It influences how we divide a problem into smaller parts and how data and operations are organized.

Four paradigms mentioned in the syllabus:

- **Procedural programming:** focuses on procedures/functions and the sequence of operations. The problem is usually broken into functions.
- **Object-oriented programming:** focuses on objects containing state (data) and behavior (methods).
- **Functional programming:** emphasizes functions, expressions and transformations of data, with strong preference for avoiding unwanted state changes.
- **Logic/rule-based programming:** represents knowledge as facts and rules and derives results from them.

Simple analogy
Procedural: “First do A, then B, then C.”
OOP: “These are the objects; each object knows its data and what it can do.”
Functional: “Apply these functions to transform the input.”
Logic/rule based: “If these facts/rules are true, derive this result.”

2. Need for Object-Oriented Programming

As software becomes larger, managing many variables and functions independently becomes difficult. OOP provides a way to group related data and operations into meaningful units.

- Improves organization of large programs.
- Models real-world entities naturally.
- Supports code reuse through classes and inheritance.
- Encourages controlled access to data.
- Makes maintenance and modification easier.
- Supports extensibility through polymorphism.
- Makes teams able to work on different classes/components independently.

Real-life example
In a college management system, a **Student** can have rollNo, name and marks, and methods such as displayDetails() or calculateGrade(). A **Course** can have courseCode, title and credits. OOP lets us represent these as separate classes.

3. Fundamentals of OOP

- **Class:** A blueprint/template describing the data and behavior that objects of that type will have.
- **Object:** A runtime instance of a class. It has state, behavior and identity.

- **Encapsulation:** Combining data and the methods that operate on it inside a class, while controlling access.
- **Abstraction:** Showing essential features while hiding unnecessary implementation details.
- **Inheritance:** Creating a new class based on an existing class so that suitable members can be reused/extended.
- **Polymorphism:** Allowing one interface/reference or method name to represent different behaviors in different contexts.
- **Message:** A request sent to an object, conceptually represented by invoking one of its methods.

4. Namespaces

A namespace is a naming mechanism used to organize names and avoid collisions. In Java, the closest practical mechanism is the **package**. A package groups related classes/interfaces and helps prevent naming conflicts.

For example, two packages can contain classes with the same simple name. Their fully qualified names remain different.

```
package college.students;

public class Student {
    // ...
}
```

The fully qualified name is **college.students.Student**. Java code can import a class from another package.

```
import java.util.Scanner;

class Demo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number:");
        int n = sc.nextInt();
        System.out.println(n);
        sc.close();
    }
}
```

Exam point
Java does not use a C++-style 'namespace' keyword. In Java, **packages** provide namespace-like organization.

5. Objects and Classes

A **class** defines what an object contains and can do. An **object** is created from that class.

```
class Student {
    String name;
    int rollNo;

    void display() {
        System.out.println(name + " " + rollNo);
    }
}

class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Asha";
        s1.rollNo = 101;
        s1.display();
    }
}
```

- **State:** values of fields such as name and rollNo.
- **Behavior:** operations such as display().
- **Identity:** the fact that s1 refers to a particular object instance.

6. Data Members, Methods and Messages

Data members/fields store object or class state. **Methods** define behavior. A method call can be viewed as sending a message/request to an object.

```
class BankAccount {
    double balance;                 // field

    void deposit(double amount) {   // method
        balance = balance + amount;
    }
}

BankAccount a = new BankAccount();
a.deposit(500);                    // message/request to object a
```

7. Data Encapsulation and Information Hiding

Encapsulation means keeping related state and behavior together in a class. A common Java practice is to make fields private and expose controlled public methods.

Information hiding means preventing other code from depending directly on internal implementation details.

```
class Account {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}
```

Why private?
If balance were public, any code could assign an invalid value directly. A method such as deposit() can enforce rules before changing the state.

8. Data Abstraction

Abstraction focuses on **what** an object provides rather than **how** it performs every internal step.

Example: when a user calls Scanner.nextInt(), the user does not need to know the internal implementation of input parsing. The public method is the usable abstraction.

Encapsulation vs Abstraction
Encapsulation: bundles data/behavior and controls access.
Abstraction: exposes essential operations and hides unnecessary implementation details.

9. Inheritance

Inheritance allows a subclass to acquire accessible members of a superclass and then add or specialize behavior.

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}

class Car extends Vehicle {
    void drive() {
        System.out.println("Car is driving");
    }
}

class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.start();
```

```
        c.drive();
    }
}
```

- Superclass/base/parent: Vehicle.
- Subclass/derived/child: Car.
- Java classes use the **extends** keyword for class inheritance.

Important
Java supports single, multilevel and hierarchical class inheritance, but a class cannot extend multiple classes. Multiple inheritance of type can be achieved through interfaces.

10. Polymorphism

Polymorphism literally means 'many forms'. In Java, common examples are **method overloading** (compile-time polymorphism) and **method overriding with dynamic method dispatch** (runtime polymorphism).

```
class Printer {
    void print(int x) {
        System.out.println("Integer: " + x);
    }

    void print(String x) {
        System.out.println("String: " + x);
    }
}
```

The same method name print() works with different parameter types. The compiler chooses the matching overloaded method.

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog sound");
    }
}

Animal a = new Dog();
a.sound();     // Dog's overridden method is selected at runtime
```

Key distinction
Overloading: same method name, different parameter list, usually within the same class.
Overriding: subclass provides a new implementation for an inherited instance method with a compatible signature.

11. Benefits of OOP

- Modularity — systems can be divided into classes/components.
- Reusability — existing classes/components can be reused.
- Maintainability — changes can often be localized.
- Security/data protection — access modifiers help control state.
- Extensibility — new subclasses and implementations can be added.
- Scalability of design — suitable for larger software systems.
- Real-world modeling — entities can be represented as objects.

12. Java as an Object-Oriented Programming Language

Java is a class-based, object-oriented language designed with portability and safety features. Programs are compiled to bytecode that runs on a Java Virtual Machine (JVM).

- Classes and objects are central to Java program design.
- Supports encapsulation using access modifiers.
- Supports inheritance between classes.
- Supports polymorphism through overloading, overriding and dynamic dispatch.
- Provides a large standard class library/API.
- Automatic memory management is provided through garbage collection.

13. Overview of Java Program Structure

```
package myapp;              // optional package declaration

import java.util.Scanner;   // optional imports

public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

- **Package declaration:** places the class in a package.
- **Import:** makes types available by simple name.
- **Class definition:** contains fields, constructors and methods.
- **main():** common entry point for a standalone Java application.
- **Statements:** executable instructions inside methods.

14. Java Packages, JVM and JDK/JRE

Package: organizes related types and helps manage access and naming.

JVM (Java Virtual Machine): executes Java bytecode. The JVM is the runtime engine that enables the same bytecode to run on different systems that provide a compatible JVM.

JDK (Java Development Kit): development kit containing tools needed to develop Java programs, including the compiler.

JRE: historically used to describe a runtime distribution containing the JVM and libraries; modern JDK distributions are generally the practical installation for developers.

Flow to remember
Java source (.java) → **javac compiler** → **bytecode (.class)** → **JVM** → execution

15. Data Types and Variables

A variable is a named memory location/reference used to store a value. Java is statically typed: a variable has a declared type.

```
int age = 20;
double salary = 45000.50;
char grade = 'A';
boolean passed = true;
String name = "Riya";
```

Java data types are broadly divided into **primitive** and **reference** types.

16. Primitive vs Reference Types

Primitive	Reference

byte, short, int, long, float, double, char, boolean	Classes, arrays, interfaces, enums, etc.
Variable directly represents a primitive value.	Variable holds a reference to an object/array (or null).
Examples: int x = 10;	Example: Student s = new Student();

Common primitive sizes (Java specification): byte 8-bit, short 16-bit, int 32-bit, long 64-bit; float 32-bit, double 64-bit; char 16-bit unsigned UTF-16 code unit; boolean has true/false values. The exact memory representation used by an implementation may include additional runtime details.

17. Floating-Point Numbers

Java provides **float** and **double** for floating-point values. A decimal literal such as 3.14 is a double by default.

```
double pi = 3.141592653589793;
float rate = 5.5f;       // f suffix is needed for a float literal
```

Accuracy warning
Binary floating-point types are approximate for many decimal fractions. For money calculations where exact decimal rounding is required, classes such as BigDecimal are often preferable to float/double.

18. Operators and Expressions

- **Arithmetic:** + - * / %
- **Relational:** == != > < >= <=
- **Logical:** && || !
- **Assignment:** = += -= *= /= %=
- **Unary:** ++ -- + - !
- **Conditional:** condition ? value1 : value2
- **Bitwise/shift:** & | ^ ~ << >> >>>

```
int a = 10, b = 3;
System.out.println(a + b);   // 13
System.out.println(a / b);   // 3 (integer division)
System.out.println(a % b);   // 1
System.out.println(a > b);   // true
```

Common trap
When both operands are integers, / performs integer division. For example, 5 / 2 gives 2, not 2.5. Using 5.0 / 2 gives 2.5.

19. Java Class Libraries

Java's standard libraries provide reusable classes and interfaces for common tasks. Examples include:

- **java.lang:** String, Math, Object, System and other fundamental classes (java.lang is automatically imported).
- **java.util:** Scanner, ArrayList, Collections, Random and more.
- **java.io:** stream and file I/O classes.
- **java.time:** modern date/time API.
- **java.math:** BigInteger and BigDecimal.

Using libraries reduces the need to implement common functionality from scratch.

20. Typical Java Development Environment and Memory Concepts

Typical development flow: write source → compile → run → test/debug → refine.

Java runtime memory is managed by the JVM. At a high level, object instances are typically allocated in a heap area, while each thread has its own stack used for method frames and local variables. The JVM specification defines runtime data areas; implementation details can vary.

Reference variable example
In **Student s = new Student();**, 's' is a reference variable. The Student object is created by new, and s refers to that object. The reference itself is not the same thing as the object.

21. Control Statements

A. Selection Statements

if: executes a block when a condition is true.

```
if (marks >= 40) {
    System.out.println("Pass");
}
```

if-else: chooses between two alternatives.

```
if (marks >= 40)
    System.out.println("Pass");
else
    System.out.println("Fail");
```

Nested if: an if statement placed inside another decision block.

```
if (age >= 18) {
    if (hasId) {
        System.out.println("Allowed");
    }
}
```

switch: selects among multiple cases based on an expression.

```
int day = 2;

switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    default:
        System.out.println("Invalid day");
}
```

Switch note
The break statements above prevent fall-through into later cases. Modern Java also supports switch expressions, but the traditional switch statement shown here is useful for syllabus fundamentals.

B. Iteration Statements

while: condition is checked before each iteration.

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

do-while: body executes at least once because the condition is checked after the body.

```
int i = 1;
do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

for: compact loop for initialization, condition and update.

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

for-each: convenient traversal of arrays/iterable collections.

```
int[] marks = {70, 82, 91};

for (int mark : marks) {
    System.out.println(mark);
}
```

break: terminates the nearest loop or switch. **continue:** skips the rest of the current loop iteration.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) continue;
    if (i == 8) break;
    System.out.println(i);
}
```

22. Unit I Case Study and Classroom Activities

Case study: College Management System
Identify objects such as Student, Teacher, Course and Department. For each object, list state (fields) and behavior (methods). Then identify where encapsulation, inheritance and polymorphism could be useful.

Activity: OOP vs Real Life
Groups get one entity—UPI app, food-delivery order, library book, railway ticket or gaming character. Each group writes: class name, 4 fields, 4 methods, one rule for data hiding and one possible inheritance relationship.

Quick classroom quiz
1) Is a class an object? 2) What is the difference between encapsulation and abstraction? 3) Why is 5/2 equal to 2 in integer arithmetic? 4) Which Java mechanism organizes classes into named groups? 5) What does JVM execute?

Unit I exam focus
Prepare definitions with examples for class, object, encapsulation, abstraction, inheritance and polymorphism. Also practice Java program structure, data types, operators, if/switch and all major loops.

UNIT II — INTRODUCTION TO CLASSES, OBJECTS AND ARRAYS

Syllabus coverage: Defining classes; fields; methods; object creation; accessing members; multiple parameters; argument passing; object parameters; returning objects; reference variables; set/get methods; constructors; this; static methods; scope; method overloading; Java API packages; arrays; passing arrays to methods; multidimensional arrays; variable-length argument lists; command-line arguments; banking/library case study.

Unit II idea
Unit I introduced the OOP mindset. Unit II now turns that mindset into Java code: you will create classes, construct objects, call methods, pass data, return objects and work with arrays.

23. Defining a Class

A class is declared using the **class** keyword. A class body can contain fields, constructors, methods, nested types and other declarations.

```
class Student {
    int rollNo;
    String name;

    void display() {
        System.out.println(rollNo + " " + name);
    }
}
```

The class definition itself does not create a Student object. An object is created separately using new.

24. Field Declaration

A field is a variable declared inside a class. It may be an instance field or a static/class field.

```
class Student {
    private int rollNo;          // instance field
    private String name;         // instance field
    static String college = "ABC"; // static field
}
```

- Instance fields belong to individual objects.
- Static fields belong to the class and are shared among instances.

25. Method Declaration and Definition

A method declaration/definition specifies access modifier (optional), return type, method name, parameter list and body.

```
public int add(int a, int b) {
    return a + b;
}
```

- **public:** access modifier.
- **int:** return type.
- **add:** method name.
- **int a, int b:** formal parameters.
- **return:** sends a value back to the caller.

26. Instantiating an Object

```
Student s1 = new Student();
```

This statement can be understood as: declare a reference variable s1 of type Student, create a Student object with new, and store the reference to that object in s1.

Remember
The **new** operator creates an object (and invokes an appropriate constructor). The variable on the left is a reference variable.

27. Accessing Class Members

```
class Student {
    String name;

    void display() {
        System.out.println(name);
    }
}

Student s = new Student();
s.name = "Neha";
s.display();
```

The dot operator (.) is used to access accessible members through an object/reference. Private members cannot be accessed directly from unrelated classes.

28. Methods with Multiple Parameters

```
class Calculator {
    int add(int a, int b, int c) {
        return a + b + c;
    }
}

Calculator c = new Calculator();
int result = c.add(10, 20, 30);
System.out.println(result);
```

The order and types of arguments in the call must match a compatible method signature.

29. Argument Passing

Java uses **pass-by-value**. For primitive arguments, the value is copied. For object references, the reference value is copied; therefore a method can modify the state of the referenced object, but it cannot make the caller's reference variable point to a different object.

```
class Demo {
    static void change(int x) {
        x = 100;
    }

    public static void main(String[] args) {
        int n = 10;
        change(n);
        System.out.println(n); // 10
    }
}
```

Very important
Java is not pass-by-reference. When an object is passed, the copied reference still refers to the same object, so changes to that object's fields can be visible to the caller.

30. Object as a Parameter

```
class Student {
    int marks;

    Student(int marks) {
        this.marks = marks;
    }
}

class Result {
    void compare(Student a, Student b) {
        if (a.marks > b.marks)
```

```
            System.out.println("A has higher marks");
        else
            System.out.println("B has higher or equal marks");
    }
}
```

Objects can be passed as method arguments using the class type as the parameter type.

31. Returning Objects

```
class Point {
    int x, y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    Point shift(int dx, int dy) {
        return new Point(x + dx, y + dy);
    }
}

Point p1 = new Point(2, 3);
Point p2 = p1.shift(5, 4);
```

A method can return an object reference. The return type is the class type, such as Point.

32. Object Reference Variables

```
Student a = new Student();
Student b = a;
```

After the second statement, both a and b refer to the same Student object. Changing the object through b can be observed through a.

```
b.name = "Kavya";
System.out.println(a.name);  // Kavya
```

Aliasing
When multiple reference variables refer to the same object, they are aliases for that object. This is different from creating a second object with new.

33. set/get Methods

Getter methods read a field; setter methods update a field. They are commonly used with private fields to implement controlled access.

```
class Student {
    private int marks;

    public void setMarks(int marks) {
        if (marks >= 0 && marks <= 100) {
            this.marks = marks;
        }
    }

    public int getMarks() {
        return marks;
    }
}
```

The setter can validate input before changing state.

34. Constructors

A constructor initializes an object when it is created. It has the same name as the class and has **no return type**, not even void.

```
class Student {
    int rollNo;
```

```
    String name;

    Student() {
        rollNo = 0;
        name = "Unknown";
    }

    Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }
}
```

- A class may have multiple constructors with different parameter lists.
- If you declare no constructor, Java provides a default constructor with the same access level as the class declaration, subject to the normal rules; it initializes fields to their default values.
- If you declare any constructor, the compiler does not automatically add a no-argument constructor.

Constructor vs method
Constructor: initializes a newly created object; same name as class; no return type; invoked during object creation. Method: represents behavior; has a return type (or void); can be called after creation.

35. this Keyword

The keyword **this** refers to the current object in an instance context.

```
class Student {
    private int rollNo;

    Student(int rollNo) {
        this.rollNo = rollNo;
    }
}
```

Here, the parameter rollNo and the instance field rollNo have the same name. this.rollNo clearly refers to the field.

this can also be used to call another constructor in the same class, but a this(...) constructor invocation must be the first statement of the constructor.

```
class Box {
    int length, width;

    Box() {
        this(1, 1);
    }

    Box(int length, int width) {
        this.length = length;
        this.width = width;
    }
}
```

36. Static Methods and Members

A static member belongs to the class rather than to an individual object.

```
class Counter {
    static int count = 0;

    Counter() {
        count++;
    }

    static int getCount() {
        return count;
    }
}

Counter a = new Counter();
Counter b = new Counter();
```

```
System.out.println(Counter.getCount()); // 2
```

- Access static members using the class name when possible: Counter.getCount().
- A static method does not have a current object (no implicit this), so it cannot directly access instance fields/methods.

Easy memory trick
Instance = “each object has its own copy.”
Static = “one class-level member shared by the class.”

37. Scope of Declaration

Scope means the region of a program in which a declared name can be directly used.

- **Local variable:** declared inside a method/block; available within that scope.
- **Parameter:** available within the method/constructor declaration.
- **Instance field:** belongs to an object and can be accessed according to its access level.
- **Static field:** belongs to the class and can be accessed according to its access level.

```
class Demo {
    int x = 10;             // instance field

    void show(int p) {      // p is a parameter
        int y = 20;         // local variable
        System.out.println(x + p + y);
    }
}
```

38. Method Overloading

Method overloading means defining multiple methods with the same name but different parameter lists. Changing only the return type is not sufficient for overloading.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Valid vs invalid
Valid: add(int,int) and add(int,int,int).
Valid: add(int,int) and add(double,double).
Invalid: two methods with identical parameter lists but different return types only.

39. Java API Packages

The Java API contains standard packages with ready-to-use classes. Some frequently used packages are:

- java.lang — core language classes.
- java.util — utilities and collections.
- java.io — input/output streams and file-related classes.
- java.time — date and time.
- java.math — arbitrary-precision arithmetic.

```
import java.util.Scanner;
import java.util.Arrays;
```

Imports affect how a type can be referred to by its simple name; they do not copy the class into your program.

40. Arrays in Java — Declaring and Creating

An array is a fixed-size, indexed collection of elements of the same type. Java arrays are objects.

Declaration:

```
int[] marks;
String[] names;
```

Creation:

```
marks = new int[5];
names = new String[3];
```

Declaration + creation + initialization:

```
int[] marks = {72, 81, 65, 90};
```

- Array indexing starts at 0.
- For an array of length n, valid indices are 0 through n-1.
- The length is fixed after the array is created.
- Use array.length to obtain its length.

41. Examples Using Arrays

```
int[] marks = {72, 81, 65, 90, 88};

int sum = 0;
for (int mark : marks) {
    sum += mark;
}

double average = (double) sum / marks.length;
System.out.println("Average = " + average);
```

Another example—finding the largest value:

```
int[] a = {12, 7, 25, 4, 19};

int max = a[0];

for (int i = 1; i < a.length; i++) {
    if (a[i] > max) {
        max = a[i];
    }
}

System.out.println("Maximum = " + max);
```

42. Passing Arrays to Methods

Because arrays are objects, an array reference can be passed to a method.

```
static int sum(int[] a) {
    int total = 0;

    for (int x : a) {
        total += x;
    }

    return total;
}

public static void main(String[] args) {
    int[] values = {10, 20, 30};
    System.out.println(sum(values));
}
```

A method can also modify array elements through the received reference.

```
static void doubleValues(int[] a) {
    for (int i = 0; i < a.length; i++) {
        a[i] *= 2;
    }
}
```

Important
The reference value is passed by value, but the copied reference points to the same array object. Therefore, element changes made inside the method are visible through the original reference.

43. Multidimensional Arrays

Java supports arrays of arrays. A two-dimensional array is commonly used to represent a table/matrix.

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};

for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

The first index identifies the row and the second identifies the column. Java also allows jagged arrays, where different rows can have different lengths.

```
int[][] jagged = new int[3][];
jagged[0] = new int[2];
jagged[1] = new int[4];
jagged[2] = new int[1];
```

44. Variable-Length Argument Lists (varargs)

Varargs allow a method to accept zero or more arguments of a specified type. Syntax uses **type... name**. Inside the method, the varargs parameter behaves like an array.

```
static int sum(int... values) {
    int total = 0;

    for (int x : values) {
        total += x;
    }

    return total;
}

System.out.println(sum(10, 20));
System.out.println(sum(1, 2, 3, 4, 5));
System.out.println(sum());
```

- Only one variable-arity parameter is permitted in a method declaration.
- The varargs parameter must be the last parameter.

```
static void printStudent(String name, int... marks) {
    System.out.println(name);
    for (int m : marks) {
        System.out.println(m);
    }
}
```

45. Command-Line Arguments

Arguments supplied when a Java program is launched are received in String[] args of main().

```
public class Main {
    public static void main(String[] args) {
```

```
        System.out.println("Number of arguments: " + args.length);

        for (String s : args) {
            System.out.println(s);
        }
    }
}
```

If the program is run with arguments such as **Java Main Pune 2026**, the strings are received in args[0], args[1], args[2]. Numeric input must be converted from String, for example Integer.parseInt(args[0]).

```
public class Add {
    public static void main(String[] args) {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);

        System.out.println("Sum = " + (a + b));
    }
}
```

46. Unit II Case Study — Interactive Banking System

A good case study combines class, object, encapsulation, constructors, methods and arrays/collections.

```
class BankAccount {
    private int accountNo;
    private String holderName;
    private double balance;

    BankAccount(int accountNo, String holderName, double balance) {
        this.accountNo = accountNo;
        this.holderName = holderName;
        this.balance = balance;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            return true;
        }
        return false;
    }

    public double getBalance() {
        return balance;
    }

    public void display() {
        System.out.println(accountNo + " | " + holderName
                           + " | " + balance);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount a = new BankAccount(101, "Asha", 5000);

        a.deposit(1000);
        a.withdraw(750);
        a.display();
    }
}
```

This example demonstrates private fields, constructor initialization, public methods, validation, object creation and method calls.

Extension activity
Ask students to add: account type, transferTo(BankAccount other, double amount), a static bankName, getters, and an array of BankAccount objects. Then design a menu-driven console program.

47. Unit II Case Study — Library Management System

```
class Book {
    private int id;
    private String title;
    private boolean issued;

    Book(int id, String title) {
        this.id = id;
        this.title = title;
        this.issued = false;
    }

    public void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println(title + " issued.");
        } else {
            System.out.println("Book already issued.");
        }
    }

    public void returnBook() {
        issued = false;
    }

    public void display() {
        System.out.println(id + " | " + title
                           + " | issued = " + issued);
    }
}
```

Students can extend this into an array of Book objects and implement search by ID/title.

48. Unit II Revision and Practice

Must-know definitions
Class, object, field, method, constructor, reference variable, this, static, method overloading, array, multidimensional array, varargs and command-line arguments.

Must-write programs
1) Class with fields and methods. 2) Constructor overloading. 3) Getter/setter with validation. 4) Method overloading. 5) Object passed to a method. 6) Method returning an object. 7) Sum/maximum of array. 8) 2-D matrix traversal. 9) Varargs method. 10) Command-line addition.

Debugging checklist
Check class/file naming when a class is public; braces and semicolons; constructor name; missing return statements; array bounds; null references; static vs instance access; correct parameter types; integer division; and spelling/case of identifiers.

Mini assignment
Build a small Library or Banking application using at least two classes. Use private fields, constructors, get/set methods, method overloading, an array of objects, and at least one static member. Include a short explanation of where encapsulation and object-oriented design are used.